

TrainGuard MVP Validation Packet

Mobile-first endurance training risk app - checklist, process flow, architecture, and data model

Product definition

TrainGuard is a mobile app for endurance athletes that imports workout data from Strava, lets users complete quick recovery/pain check-ins, and generates simple training-risk alerts based on training load and body response.

Best MVP statement

The MVP imports recent Strava activities, displays them in a mobile activity feed, lets the user complete a recovery check-in after each workout, and generates simple Green/Yellow/Red training-risk alerts.

MVP validation checklist

1. User problem

- User does not want to manually enter workout distance, duration, pace, heart rate, or power.
- User already records workouts in Strava, Garmin, Apple Watch, Coros, or similar.
- User wants a simple way to understand whether training load and pain are trending in the wrong direction.
- User wants quick post-workout check-ins, not long forms.
- User wants mobile access, not a website.

2. Core user flow

- User opens mobile app.
- User creates account or logs in.
- User connects Strava.
- App imports recent activities.
- User views activity feed.
- User taps an activity.
- User completes pain/recovery check-in.
- App updates dashboard.
- App generates Green/Yellow/Red training status.

3. Data source decision

- MVP uses Strava first.
- Garmin is future phase, not MVP.
- Manual workout entry is fallback only.
- App stores imported activity data in PostgreSQL.
- Backend, not mobile app, manages Strava tokens.

4. What Strava provides

- Activity name.
- Activity type: run, ride, swim, etc.
- Date and time.
- Distance.
- Duration.
- Pace or speed.
- Elevation.
- Heart rate, if available.
- Power, if available.

5. What user manually enters

- Pain score: 0-10.
- Body area.
- Side: left, right, both.
- Pain timing: before, during, after, next morning.
- Fatigue level.
- Soreness level.
- Sleep quality.
- Optional note.

6. MVP screens

- Login screen.
- Connect Strava screen.
- Home/dashboard screen.
- Activity feed screen.
- Activity detail screen.
- Recovery check-in screen.
- Alerts screen.
- Settings screen.

Dashboard validation

Dashboard should answer: Can I train normally, adjust, or recover?

- Current training status: Green, Yellow, Red, Blue.
- Weekly run mileage.
- Weekly bike hours.
- Total training hours.
- Highest pain score.
- Recent alert.
- Latest activity needing check-in.

Alert logic validation

- Weekly run mileage increased more than 15% compared to recent average.
- Long run increased more than 20%.
- Pain score is 4/10 or higher.
- Pain increased across multiple check-ins.
- More than 2 hard efforts occurred in 7 days.
- Bike volume increased sharply.
- Pain plus load increase creates a higher-risk alert.

MVP boundaries

Do not build these in version 1:

- Garmin integration.
- Apple Health integration.
- AI coach.
- Full training plan generator.
- Social features.
- Payments.
- Web dashboard.
- Advanced machine learning.
- Coach/admin portal.

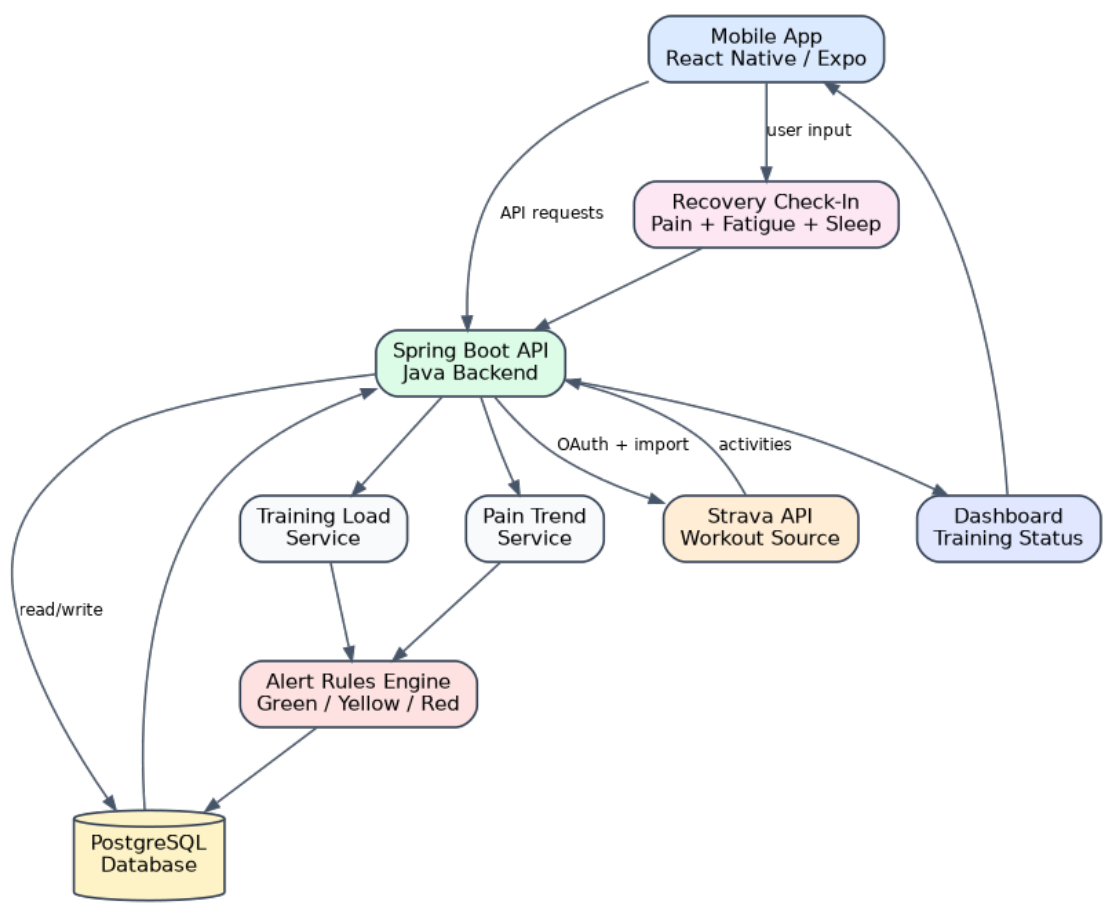
Recommended MVP decision set

- Primary user: runner/cyclist.
- Platform: mobile app only.
- Data source: Strava first.
- Backend: Spring Boot.
- Database: PostgreSQL in Docker.

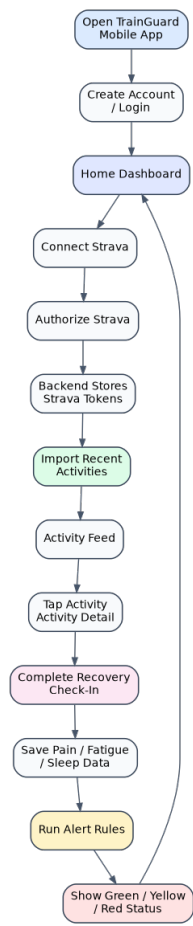
- Check-ins: tied to individual activities.
- Dashboard period: last 7 days.
- Alert levels: Green, Yellow, Red.
- Manual workout entry: later.
- Garmin: later.
- Equipment tracking: later.
- Goals: later.

Process and architecture diagrams

High-level system diagram



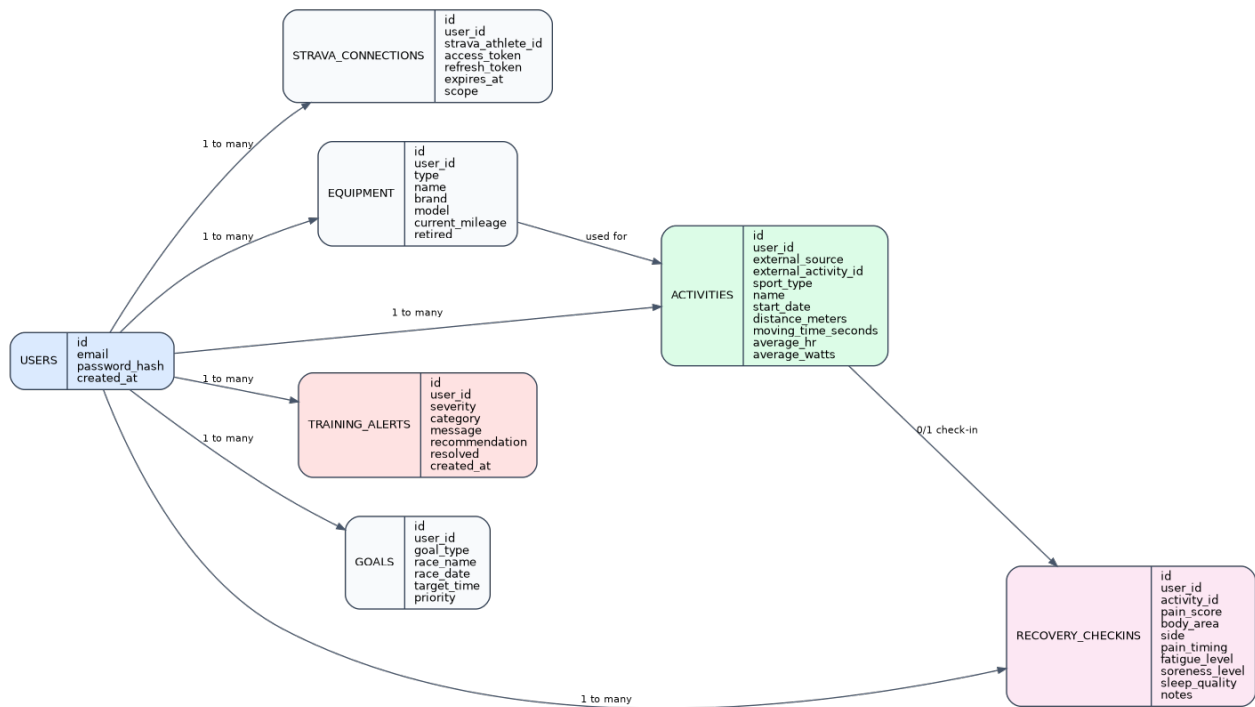
User process diagram



Backend process diagram



Database relationship diagram



MVP data flow

Step	Process
1	Strava activity data is available after user authorization.
2	Spring Boot imports and normalizes the activity data.
3	PostgreSQL stores activity history.
4	React Native app displays the activity feed.
5	User adds recovery/pain check-in.
6	Backend calculates training load and pain trend.
7	Alert engine creates Green / Yellow / Red status.
8	Mobile dashboard shows recommendation.

Key validation questions before design

- Is the primary user a runner, cyclist, or runner/cyclist?
- Should the app require login before connecting Strava?
- Should check-ins be activity-based or daily-based?
- Should pain score be required after every workout?
- Should dashboard status be based on the last 7 days or current calendar week?
- Should the app show recommendations or only alerts?
- Should users be able to dismiss/resolve alerts?
- Should the app support manual activity creation as a fallback?
- Should equipment tracking be MVP or later?
- Should goals/race tracking be MVP or later?

MVP success criteria

- Open the mobile app.
- Connect Strava.
- Import recent workouts.
- See runs and rides in an activity feed.
- Tap one activity.
- Add a recovery check-in in under 30 seconds.
- See an updated training status.
- Understand why the status is Green, Yellow, or Red.